# PySpark 语法速查表

最佳实践版 · 统一使用 F.col · Palantir Foundry 适用

## 导入与骨架

```python
# 统一用 F / T 命名空间
from pyspark.sql import functions as F, types as T
from pyspark.sql import Window

# Foundry 转换骨架
from transforms.api import transform_df, Input, Output

@transform_df(
    Output("/proj/clean"),
    src=Input("/proj/raw"),
)
def compute(src):
    return src.filter(F.col("age") > 25)
```

**提示：**不要 import *：会覆盖内置的 sum / max / min / round

## 查看数据

```python
df.show(5, truncate=False)  # 前5行，不截断
df.printSchema()            # 列名与类型
df.columns                  # 列名列表
df.count()                  # 会触发计算
df.describe("salary").show()  # 基本统计
df.explain()                # 执行计划
```

**提示：**count() / collect() / toPandas() 都是 action，大表慎用

## 选择与新增列

```python
# 选列 + 表达式 + 别名
df.select(
    F.col("name"),
    (F.col("salary") * 1.1).alias("new_salary"),
)

# 新增 / 覆盖一列
df = df.withColumn("bonus", F.col("salary") * 0.15)

# 常量列用 F.lit
df = df.withColumn("src", F.lit("hr"))

# 保留全部列并追加
df.select(
    F.col("*"),
    (F.col("a") + F.col("b")).alias("c"),
)
```

**提示：**多列变换优先一次 select，避免长串 withColumn

## 删除 / 重命名 / 类型

```python
# 删除列
df = df.drop("tmp1", "tmp2")

# 重命名
df = df.withColumnRenamed("old", "new")

# 类型转换
df = df.withColumn("age", F.col("age").cast("int"))
df = df.withColumn("d", F.col("d").cast(T.DateType()))

# 批量改列名（如全部小写）
df = df.toDF(*[c.lower() for c in df.columns])
```

## 过滤

```python
df.filter(F.col("age") > 25)

# AND 用 &，每个条件加括号
df.filter(
    (F.col("age") > 25) & (F.col("city") == "NYC")
)

# OR 用 |，NOT 用 ~
df.filter((F.col("a") == 1) | ~F.col("flag"))

# 多值匹配用 isin，比多个 OR 清晰
df.filter(F.col("dept").isin("Sales", "HR"))

df.filter(F.col("age").between(18, 65))   # 闭区间
df.filter(F.col("name").like("A%"))       # SQL 通配符
df.filter(F.col("name").rlike("^A.*n$"))  # 正则
df.filter(F.col("email").isNotNull())     # 非空

df.dropDuplicates(["email"])  # 按列去重
df.distinct()                 # 整行去重
```

**提示：**不能用 and / or / not，必须用 & | ~

## 条件与空值

```python
# CASE WHEN：务必写 otherwise，否则得 NULL
df.withColumn("level",
    F.when(F.col("sal") >= 100000, "A")
     .when(F.col("sal") >= 50000, "B")
     .otherwise("C"))

# 取第一个非空值
F.coalesce(F.col("a"), F.col("b"), F.lit(0))

# 填充空值
df.fillna({"age": 0, "name": "n/a"})

# 丢弃含空值的行
df.dropna(subset=["email"])

# 空值安全的相等比较
F.col("a").eqNullSafe(F.col("b"))
```

**提示：**NULL == NULL 的结果是 NULL，不是 True

## 字符串

```
# 去空格 + 小写
F.lower(F.trim(F.col("email")))

# 截取：从第1位起取3个（1 起始）
F.substring(F.col("s"), 1, 3)

# 拼接：任一为 NULL 则结果 NULL
F.concat(F.col("a"), F.lit("-"), F.col("b"))

# 带分隔符拼接：跳过 NULL
F.concat_ws("-", F.col("a"), F.col("b"))

# 分割并取第1段
F.split(F.col("s"), " ").getItem(0)

# 只保留数字
F.regexp_replace(F.col("p"), "[^0-9]", "")

# 正则提取（第1个分组）
F.regexp_extract(F.col("e"), "@(.*)$", 1)

F.lpad(F.col("id"), 5, "0")   # 左补零
F.length(F.col("name"))       # 长度
F.col("s").contains("x")      # 包含
```

## 日期与时间

```
# 字符串 → 日期 / 时间戳
F.to_date(F.col("s"), "yyyy-MM-dd")
F.to_timestamp(F.col("s"), "yyyy-MM-dd HH:mm:ss")

# 格式化为字符串
F.date_format(F.col("d"), "MM/dd/yyyy")

# 加减
F.date_add(F.col("d"), 7)     # +7 天
F.date_sub(F.col("d"), 7)     # -7 天
F.add_months(F.col("d"), 3)   # +3 个月

# 差值（结束，开始）
F.datediff(F.col("end"), F.col("start"))       # 天
F.months_between(F.col("end"), F.col("start")) # 月

# 提取部分
F.year(F.col("d"))
F.month(F.col("d"))
F.dayofmonth(F.col("d"))
F.dayofweek(F.col("d"))   # 1=周日

# 截断到月初
F.trunc(F.col("d"), "month")
F.date_trunc("month", F.col("ts"))

F.current_date()
F.current_timestamp()
```

## 数学

```
F.round(F.col("x"), 2)     # 四舍五入
F.floor(F.col("x"))        # 向下取整
F.ceil(F.col("x"))         # 向上取整
F.abs(F.col("x"))          # 绝对值
F.sqrt(F.col("x"))         # 平方根
F.pow(F.col("x"), 2)       # 幂
F.log(F.col("x"))          # 自然对数

# 行内比较（跨列取大 / 小）
F.greatest(F.col("a"), F.col("b"))
F.least(F.col("a"), F.col("b"))
```

**提示：**聚合用 F.max / F.min；跨列比较用 greatest / least

## 聚合

```
agg = (
    df.groupBy(F.col("dept"))
    .agg(
        F.count(F.lit(1)).alias("n"),          # 行数
        F.countDistinct(F.col("id")).alias("n_id"),
        F.sum(F.col("salary")).alias("total"),
        F.avg(F.col("salary")).alias("avg"),
        F.max(F.col("salary")).alias("max"),
        F.collect_set(F.col("name")).alias("names"),
    )
)

# HAVING = 聚合之后再 filter
agg.filter(F.col("total") > 500000)

# 条件聚合
F.sum(F.when(F.col("st") == "ok", 1).otherwise(0))

# 透视：给出取值列表更快
(df.groupBy("dept")
   .pivot("year", [2024, 2025])
   .agg(F.sum(F.col("rev"))))

# 整表聚合（不分组）
df.agg(F.max(F.col("ts")).alias("latest"))
```

## 连接与合并

```
# 同名键：结果不会出现重复列
a.join(b, on="id", how="inner")
a.join(b, on=["id", "date"], how="left")

# 键名不同：先 alias，再用 F.col
a = a.alias("a")
b = b.alias("b")
a.join(b, F.col("a.id") == F.col("b.user_id"), "left")

# how 可选
# inner  left  right  outer  cross
# left_semi  left_anti

# 广播小表
a.join(F.broadcast(small), on="id", how="left")

# 上下合并，按列名对齐
a.unionByName(b, allowMissingColumns=True)
```

**提示：**始终显式写 how；避免 right join，交换左右表即可。join 后同名列会引发 ambiguous 错误

## 窗口函数

```
w = (
    Window.partitionBy(F.col("dept"))
    .orderBy(F.col("sal").desc())
)

# 排名
F.row_number().over(w)   # 1,2,3 不重复
F.rank().over(w)         # 1,2,2,4 并列跳号
F.dense_rank().over(w)   # 1,2,2,3 并列不跳

# 每组 Top 3
(df.withColumn("rn", F.row_number().over(w))
   .filter(F.col("rn") <= 3))

# 上一行 / 下一行
F.lag(F.col("x"), 1).over(w)
F.lead(F.col("x"), 1).over(w)

# 累计和：按行的窗口
w2 = (
    Window.partitionBy(F.col("uid"))
    .orderBy(F.col("d"))
    .rowsBetween(
        Window.unboundedPreceding,
        Window.currentRow,
    )
)
F.sum(F.col("amt")).over(w2)

# 近 7 天：按秒做值范围窗口
w3 = (
    Window.partitionBy(F.col("uid"))
    .orderBy(F.col("ts").cast("long"))
    .rangeBetween(-6 * 86400, 0)
)
```

## 数组与结构

```
F.explode(F.col("tags"))          # 一元素一行（丢弃空数组）
F.explode_outer(F.col("tags"))    # 保留空 / NULL
F.size(F.col("tags"))             # 数组长度
F.array_contains(F.col("tags"), "x")   # 是否包含
F.collect_list(F.col("x"))        # 聚合成数组
F.struct(F.col("a"), F.col("b")) # 打包成结构体
F.col("addr.city")                # 取结构体字段
F.col("m")["key"]                 # 取 map 的值
```

## 排序与抽样

```
df.orderBy(F.col("a").desc(), F.col("b").asc())

# 空值排最后
df.orderBy(F.col("a").desc_nulls_last())

df.limit(10)                          # 取 10 行
df.sample(fraction=0.1, seed=42)    # 约 10%
```

**提示：**limit 只是截断，不保证顺序；要 Top N 请先 orderBy

## 输出、SQL 与日志

```
# Foundry：直接 return DataFrame

# 通用 Spark 写出
df.write.mode("overwrite").parquet("/out/path")

# 与 SQL 互通
df.createOrReplaceTempView("emp")
spark.sql(
    "SELECT dept, AVG(sal) FROM emp GROUP BY dept"
)

# Python 日志
import logging
log = logging.getLogger(__name__)
log.info("rows=%s", df.count())
```

## 完整示例

```
# 每个部门在职员工中薪资前 3
@transform_df(
    Output("/proj/top_paid"),
    emp=Input("/proj/emp"),
)
def compute(emp):
    w = (
        Window.partitionBy(F.col("dept"))
        .orderBy(F.col("sal").desc())
    )
    return (
        emp
        .filter(F.col("active"))
        .withColumn("sal", F.col("sal").cast("double"))
        .withColumn("rn", F.row_number().over(w))
        .filter(F.col("rn") <= 3)
        .select(
            F.col("dept"), F.col("name"), F.col("sal"),
        )
    )
```

## 最佳实践清单

1. 用 F.col(...) 引用列，统一用 F / T 命名空间，不写 import *
2. 多列变换用一次 select，少用长串 withColumn
3. when 链末尾一定写 otherwise，否则未命中为 NULL
4. 布尔条件用 & | ~，且每个子条件加括号
5. join 显式写 how；小表用 F.broadcast；同名列先 alias
6. 尽早 filter、只 select 需要的列，减少 shuffle 数据量
7. 优先用内置函数，避免 Python UDF（慢）
8. 大数据避免 collect() / toPandas()；count() 会触发计算
9. 生产环境显式指定 schema，少用 inferSchema
10. 记住 NULL 语义：NULL == NULL 为 NULL，用 eqNullSafe